PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI DAN KOMUNIKASI

Kurikulum Berbasis OBE (Outcome-Based Education) • Pemrograman Web

LAPORAN RESMI PRAKTIKUM

PRAKTIKUM 6: JAVASCRIPT DOM DAN EVENT HANDLING

Implementasi Pemilihan Elemen, Manipulasi DOM, Event Handling, Validasi Formulir, To-Do List Interaktif, dan Mini Project Registrasi Workshop

Nama Lengkap	:	**Margent Ouwen Sitompul**
NIM	:	**312510463**
Mata Kuliah	:	**Pemrograman Web**
Pertemuan	:	**7 (Praktikum 6)**
Repository	:	**Lab6Web**

TAHUN AKADEMIK 2026/2027

Disusun guna memenuhi syarat kelulusan modul praktikum mata kuliah Pemrograman Web

BAB I. IDENTITAS DAN CAPAIAN PRAKTIKUM

1.1 Identitas Praktikum

Mata Kuliah	Pemrograman Web
Pertemuan / Modul	Pertemuan 7 / Praktikum 6
Topik Utama	JavaScript DOM (Document Object Model) dan Event Handling
Cakupan Materi	Pemilihan dan manipulasi elemen, event listener, validasi form, dan interaksi pengguna
Nama Mahasiswa	**Margent Ouwen Sitompul**
Nomor Induk Mahasiswa (NIM)	**312510463**
Perangkat Kerja	Laptop / PC, Visual Studio Code, Browser Google Chrome, Node.js v26.7.0

1.2 Tujuan Praktikum

Berdasarkan panduan modul praktikum, tujuan yang harus dicapai mahasiswa meliputi:

1. Memahami konsep **Document Object Model (DOM)** sebagai representasi dokumen HTML yang dapat diakses dan dimanipulasi secara programatis melalui JavaScript.
2. Mampu menggunakan `id`, `class`, dan selector CSS untuk memilih elemen HTML dengan metode `getElementById()`, `querySelector()`, dan `querySelectorAll()`.
3. Mampu mengubah teks (`textContent`), atribut (`setAttribute`), kelas CSS (`classList.toggle`, `add`, `remove`), dan gaya elemen melalui JavaScript.
4. Mampu menangani aksi interaksi pengguna menggunakan `addEventListener()` pada berbagai jenis event seperti `click`, `input`, `change`, dan `submit`.
5. Mampu membuat validasi formulir sisi klien (client-side form validation) dan menampilkan pesan kesalahan/sukses sesuai kondisi input.
6. Mampu mengembangkan aplikasi web interaktif (Mini Project Form Pendaftaran Workshop) dengan minimal tiga fungsi interaktif yang berjalan dengan benar.

BAB II. DASAR TEORI

2.1 Document Object Model (DOM)

Document Object Model (DOM) adalah antarmuka pemrograman (API) standar W3C yang merepresentasikan dokumen HTML atau XML sebagai hierarki pohon objek (*DOM Tree*). Ketika browser web memuat halaman HTML, browser akan melakukan proses *parsing* kode sumber menjadi objek-objek JavaScript yang dapat dibaca, diubah, dihapus, maupun ditambahkan secara dinamis pada saat *runtime* tanpa me-reload dokumen secara keseluruhan.

2.2 Pemilihan Elemen (Element Selector)

JavaScript menyediakan beberapa metode utama untuk memilih elemen HTML:

- `document.getElementById("id")`: Memilih satu elemen spesifik berdasarkan nilai atribut `id`. Metode ini merupakan selector paling cepat karena indeks ID dioptimalkan secara langsung oleh mesin browser.
- `document.querySelector("selector")`: Memilih elemen pertama dalam dokumen yang cocok dengan selector CSS tertentu (misalnya `.class`, `#id`, `tag[attr]`).
- `document.querySelectorAll("selector")`: Memilih seluruh elemen yang cocok dengan selector CSS dalam bentuk koleksi `NodeList` statis yang dapat diiterasi menggunakan perulangan `forEach`.

2.3 Manipulasi Konten dan Aspek Keamanan

Untuk memperbarui teks pada elemen, standar modern merekomendasikan penggunaan `textContent` dibandingkan `innerHTML` saat menampilkan data masukan pengguna. `textContent` memperlakukan seluruh string sebagai teks murni (*literal string*) sehingga mencegah celah keamanan **Cross-Site Scripting (XSS)** di mana penyerang berusaha menyisipkan skrip berbahaya ke dalam DOM.

2.4 Event Handling & addEventListener()

Event adalah kejadian atau aksi yang terjadi pada browser, seperti pengguna menekan tombol mouse (`click`), mengetik huruf (`input`), mengganti pilihan dropdown (`change`), atau mengirim formulir (`submit`). Melalui `addEventListener()`, kita dapat menghubungkan suatu elemen dengan fungsi penangan (*callback listener*) secara modular tanpa menimpa event listener lainnya.

Pada event `submit` pada formulir, pemanggilan `event.preventDefault()` sangat penting untuk menghentikan aksi bawaan browser yang biasanya me-refresh halaman (*reload*), sehingga verifikasi validasi data dapat diselesaikan secara mulus melalui JavaScript.

BAB III. LANGKAH KERJA & IMPLEMENTASI LATIHAN 1–6

3.1 Persiapan Struktur Direktori & File

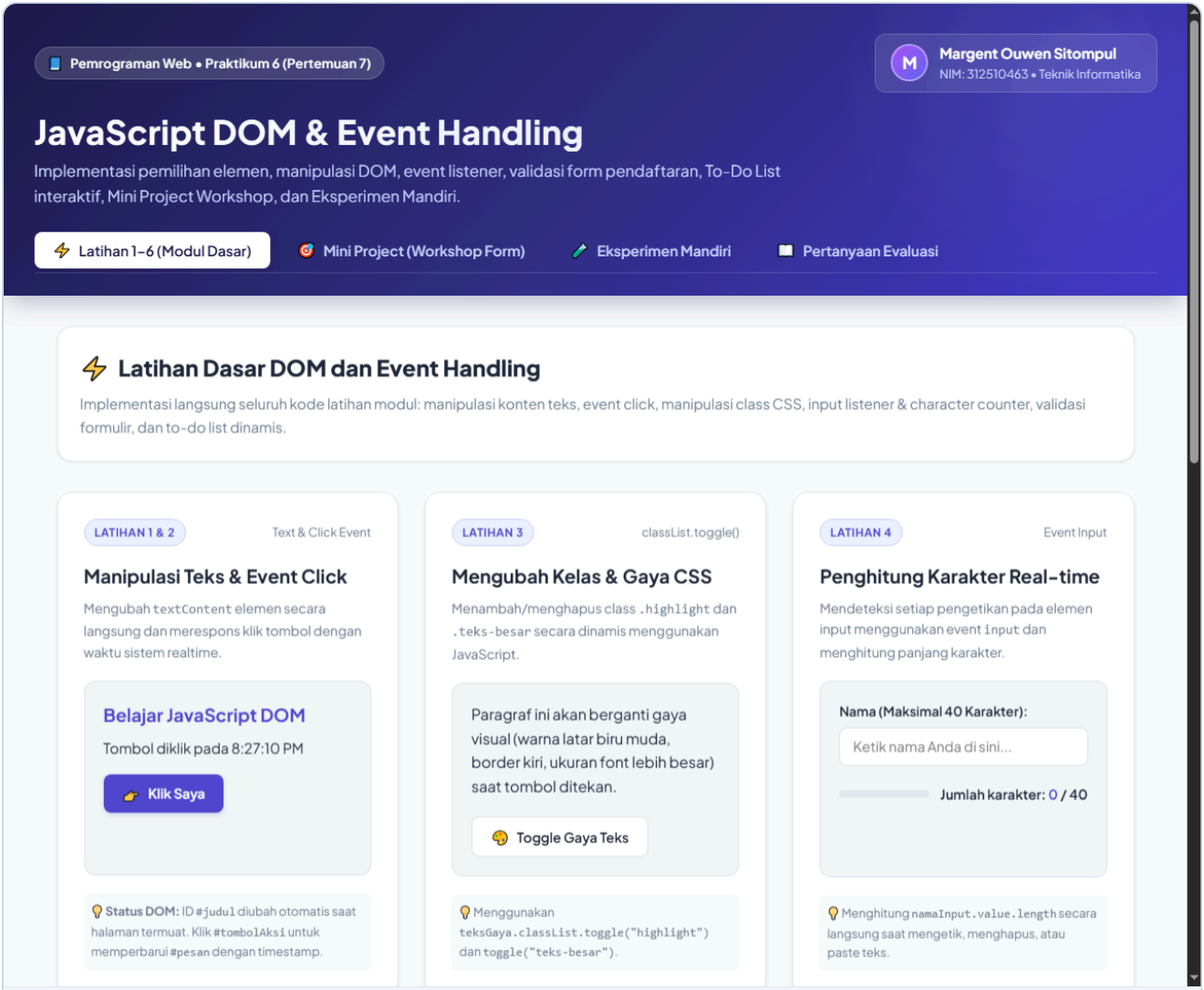
Sesuai instruksi Bagian D dan I Modul, repository dibangun bernama **Lab6Web** dengan struktur file yang rapi:

```
Lab6Web/ ├── index.html # Halaman dashboard terpadu (Latihan 1-6, Mini Project, Eksperimen, Evaluasi) ├── style.css # Desain sistem CSS modern & kelas modul (.highlight, .teks-besar, .selesai) ├── script.js # Skrip JavaScript interaktif modular ├── lab6_dom_event/ # Direktori standalone modul latihan | ├── index.html | ├── style.css | └── script.js ├── screenshots/ # Berkas dokumentasi pengujian tangkapan layar (11 berkas PNG) ├── README.md # Dokumentasi lengkap proyek ├── laporan.html # Dokumen sumber cetak PDF └── Laporan_Praktikum6_312510463_Margent_Ouwen_Sitompul.pdf
```

3.2 Latihan 1 & 2: Manipulasi Teks DOM dan Event Click

Tujuan: Mengubah teks pada elemen ber-ID `#judul` dan `#pesan` menggunakan `textContent`, serta memasang event listener `click` pada `#tombolAksi` untuk memperbarui pesan dengan waktu lokal terkini.

```
// script.js (Latihan 1 & 2) const judul = document.getElementById("judul"); const pesan = document.getElementById("pesan"); const tombolAksi = document.getElementById("tombolAksi"); // Latihan 1: Inisialisasi teks judul.textContent = "Belajar JavaScript DOM"; pesan.textContent = "Konten berhasil diubah oleh JavaScript."; // Latihan 2: Penanganan klik tombol tombolAksi.addEventListener("click", function () { pesan.textContent = "Tombol diklik pada " + new Date().toLocaleTimeString(); });
```

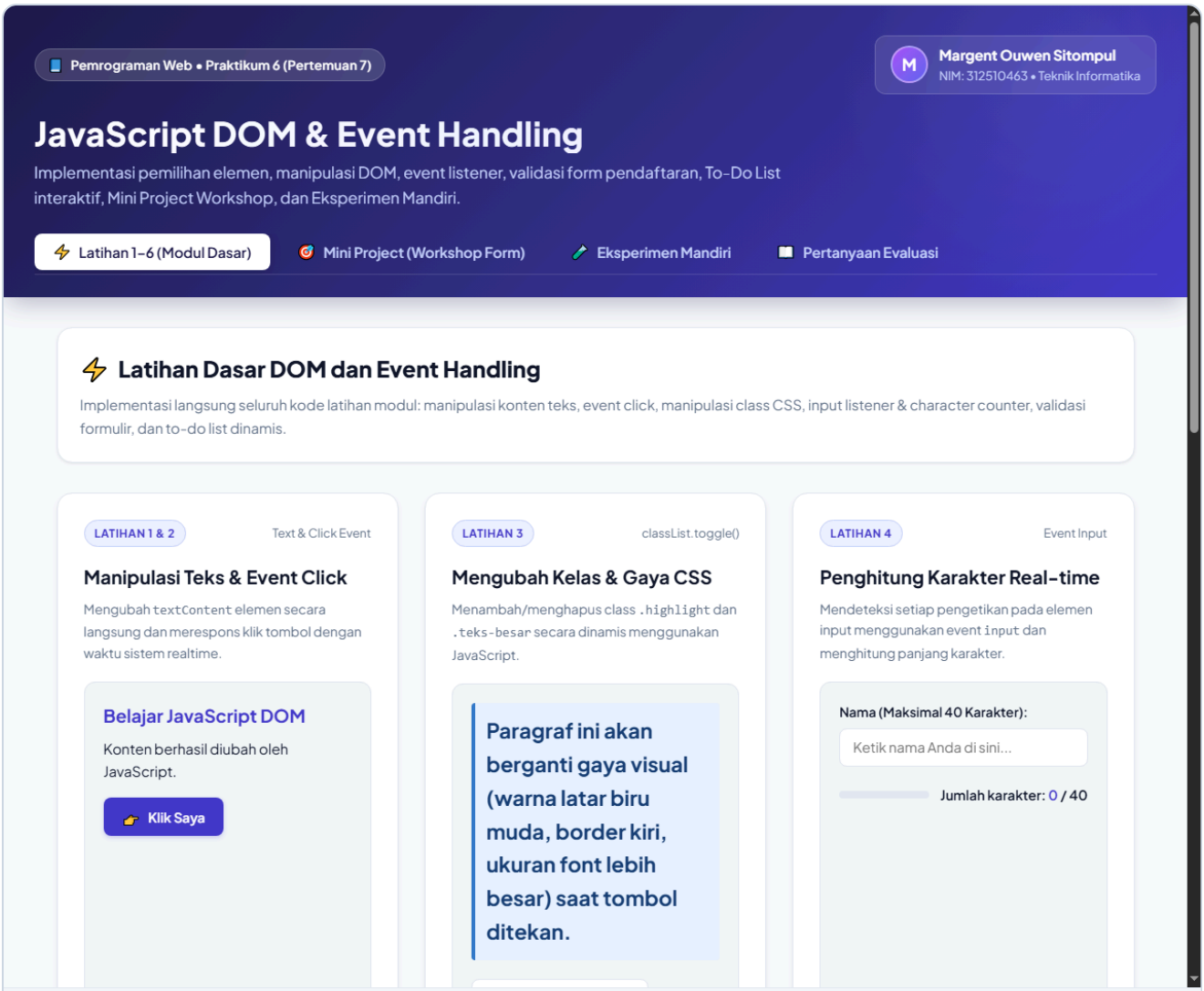


Gambar 3.1: Hasil Pengujian Latihan 1 (Teks Berubah) dan Latihan 2 (Event Click Waktu)

3.3 Latihan 3: Mengubah Kelas dan Gaya CSS (classList.toggle)

Tujuan: Mengendalikan tampilan elemen `#teksGaya` secara dinamis menggunakan metode `classList.toggle("highlight")` dan `classList.toggle("teks-besar")` saat `#tombolGaya` ditekan.

```
/* style.css (Latihan 3) */ .highlight { color: #173f75; background-color: #e6f1ff; border-left: 4px solid #3478c8; padding: 12px; } .teks-besar { font-size: 1.4rem; } // script.js (Latihan 3) const tombolGaya = document.getElementById("tombolGaya"); const teksGaya = document.getElementById("teksGaya"); tombolGaya.addEventListener("click", function () { teksGaya.classList.toggle("highlight"); teksGaya.classList.toggle("teks-besar"); });
```

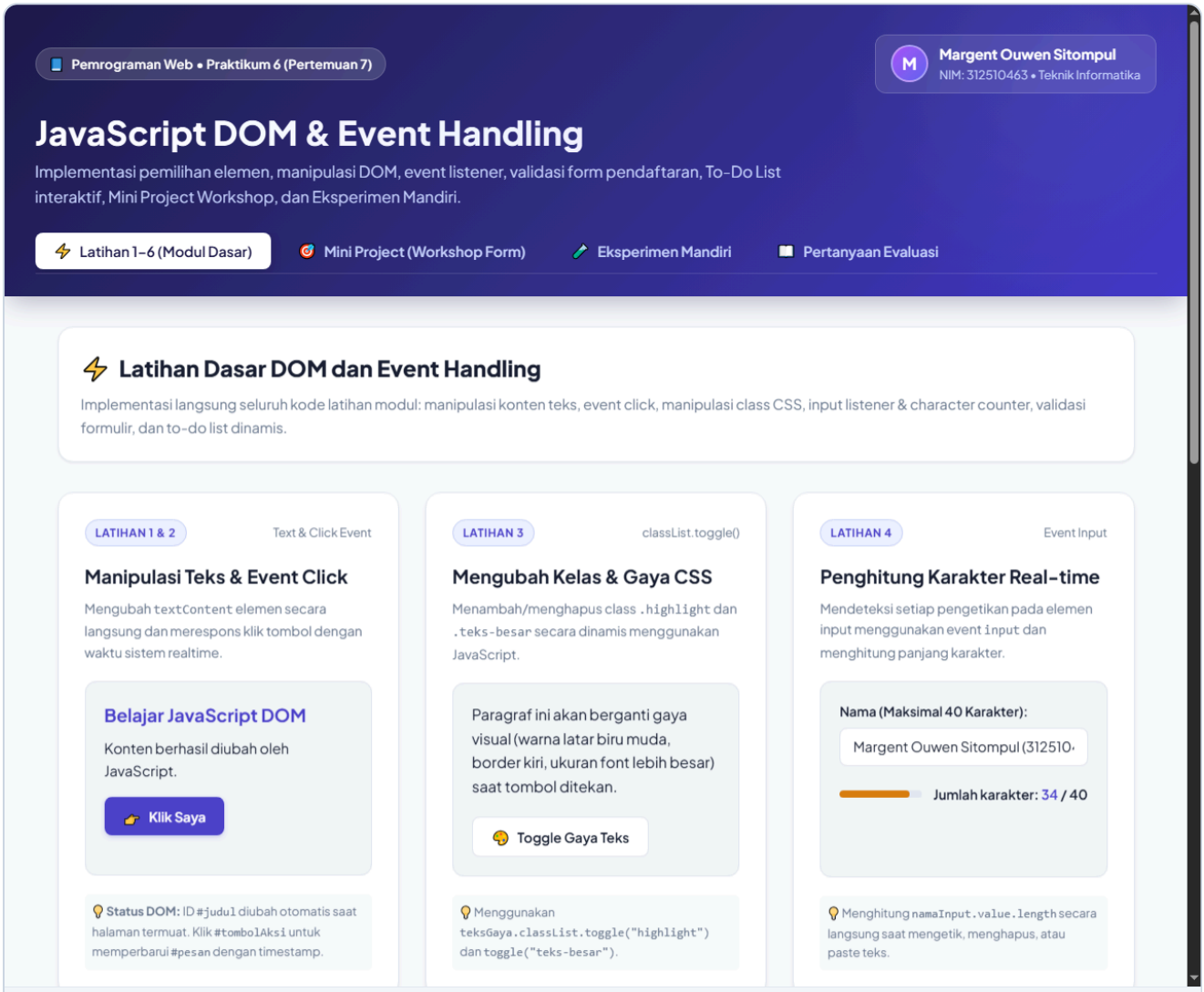


Gambar 3.2: Hasil Pengujian Latihan 3 saat Class Highlight dan Teks-Besar Diaktifkan

3.4 Latihan 4: Event Input dan Penghitung Karakter Real-time

Tujuan: Membaca input teks secara *real-time* menggunakan event `input` pada `#namaInput` dan menampilkan jumlah karakter aktif pada elemen `#jumlahKarakter`.

```
// script.js (Latihan 4) const namaInput = document.getElementById("namaInput"); const
jumlahKarakter = document.getElementById("jumlahKarakter");
namaInput.addEventListener("input", function () { jumlahKarakter.textContent =
namaInput.value.length; });
```



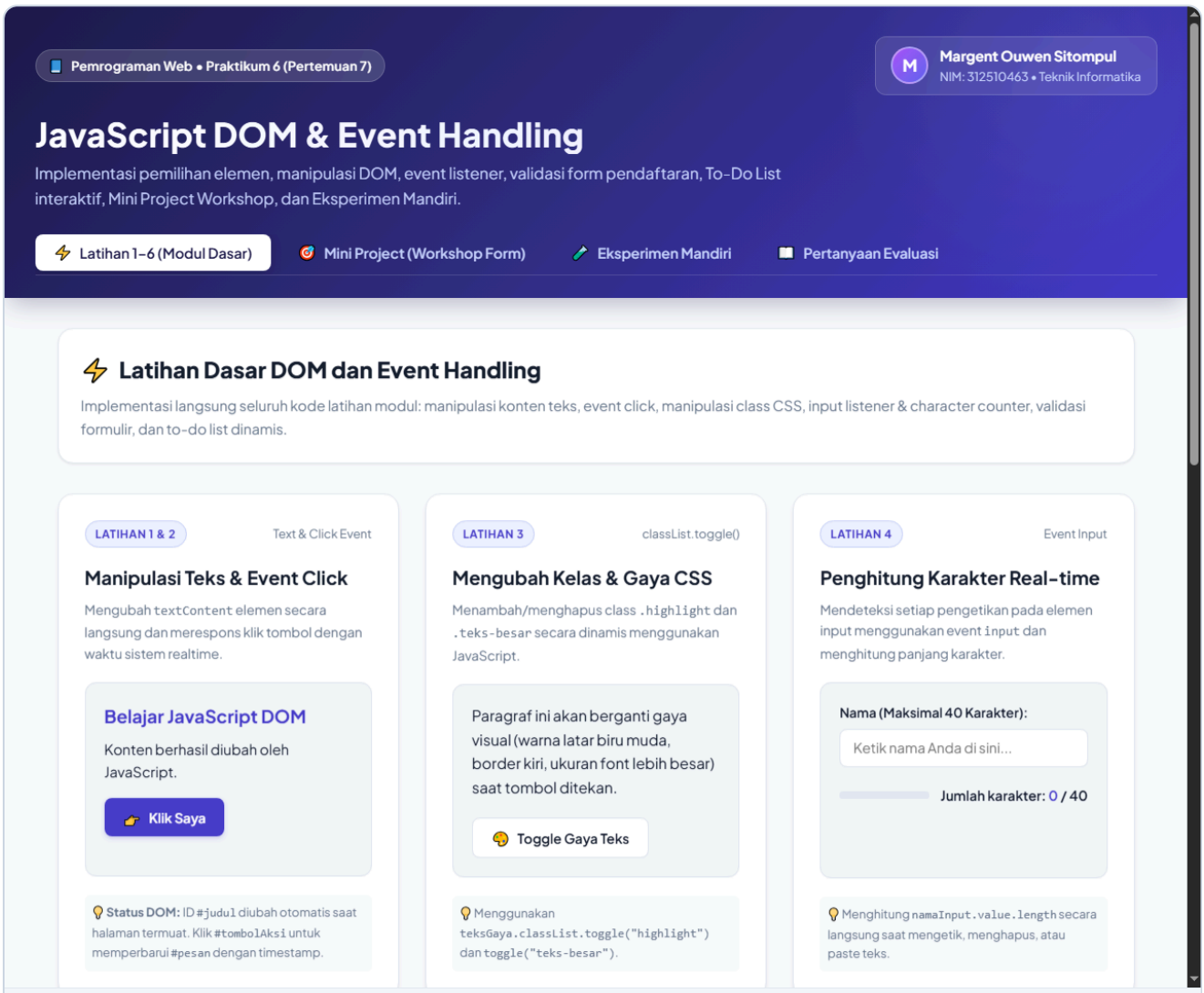
Gambar 3.3: Hasil Pengujian Latihan 4 Menampilkan Jumlah Karakter Real-time (Nama: Margent Ouwen Sitompul)

3.5 Latihan 5: Validasi Formulir Pendaftaran

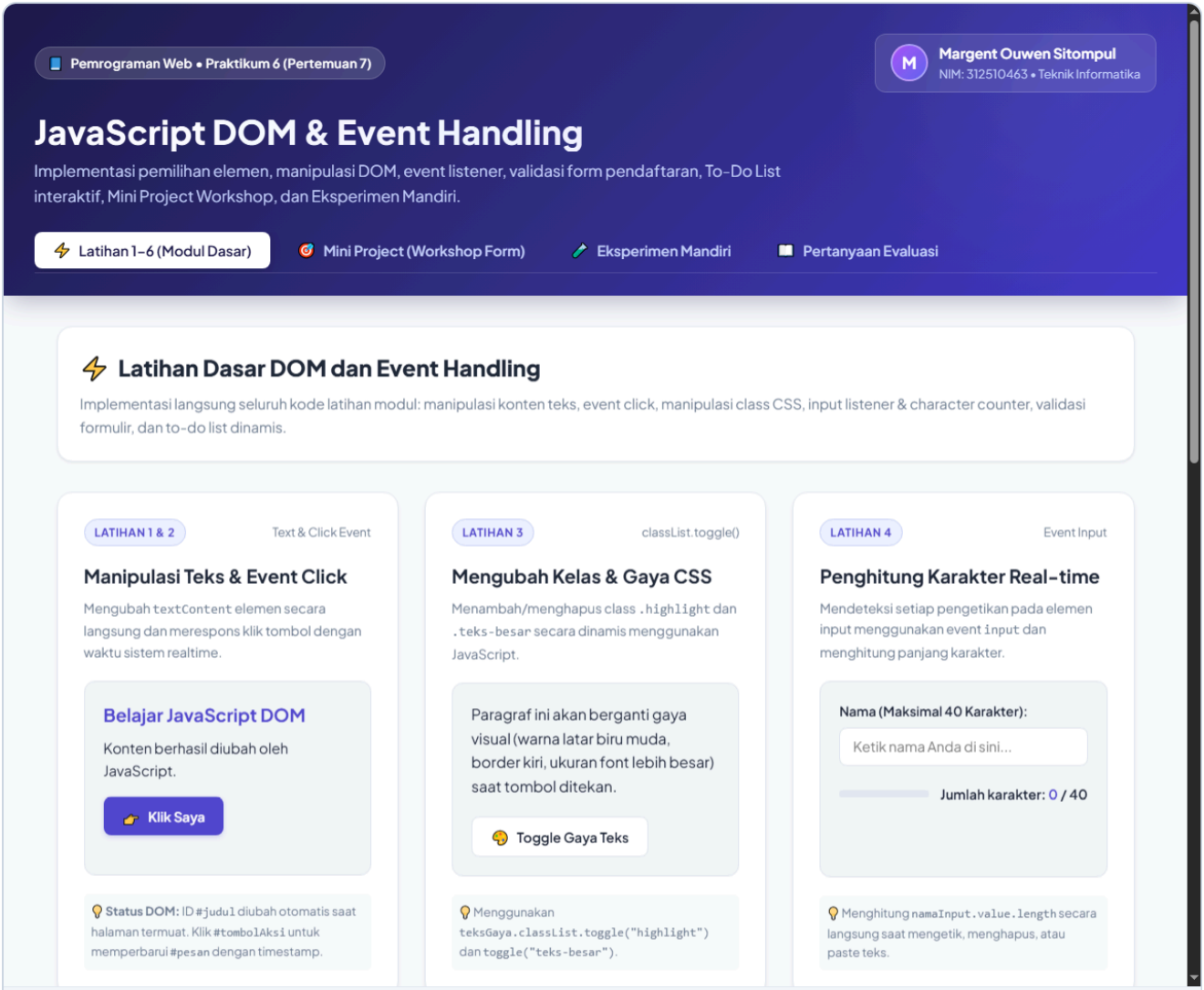
Tujuan: Memeriksa keabsahan masukan formulir pendaftaran (nama tidak boleh kosong, pola email sesuai regex, dan usia berupa bilangan bulat dalam rentang 17–100 tahun).

```
// script.js (Latihan 5) const form = document.getElementById("formPendaftaran"); const
pesanForm = document.getElementById("pesanForm"); form.addEventListener("submit", function
```

```
(event) { event.preventDefault(); // Mencegah reload halaman const nama =
document.getElementById("nama").value.trim(); const email =
document.getElementById("email").value.trim(); const usiaInput =
document.getElementById("usia").value; const usia = Number(usiaInput); const polaEmail =
/^[^\s@]+@[^\s@]+\.[^\s@]+$/; if (nama === "") { pesanForm.textContent = "Nama wajib
diisi."; } else if (!polaEmail.test(email)) { pesanForm.textContent = "Masukkan email yang
valid."; } else if (usiaInput === "" || !Number.isInteger(usia) || usia < 17 || usia > 100)
{ pesanForm.textContent = "Usia harus berupa bilangan bulat 17-100."; } else {
pesanForm.textContent = "Validasi berhasil. Form siap diproses."; form.reset(); } });
```



Gambar 3.4: Hasil Pengujian Latihan 5 – Kondisi Tidak Valid (Format Email Salah)

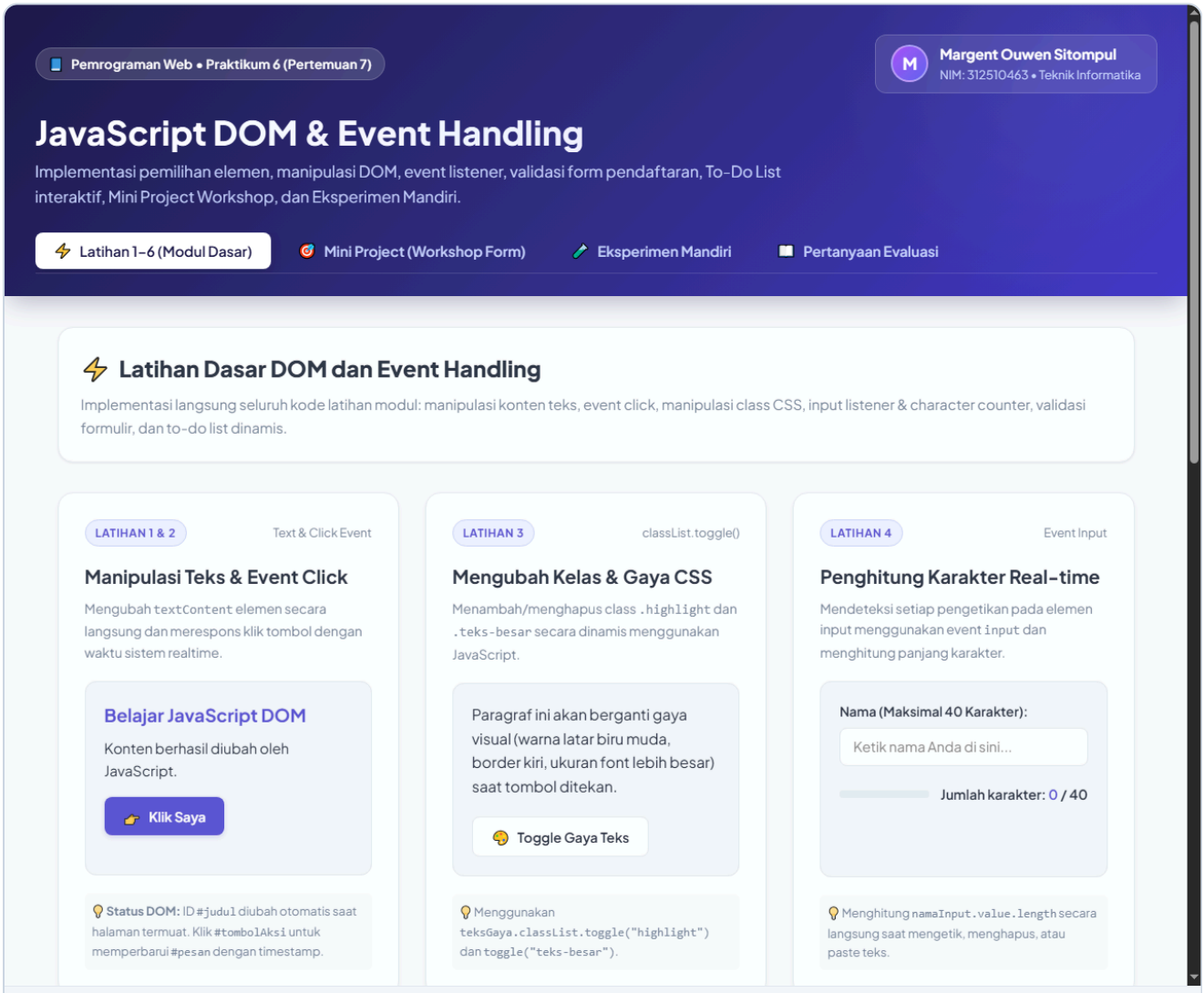


Gambar 3.5: Hasil Pengujian Latihan 5 – Kondisi Valid ("Validasi berhasil. Form siap diproses.")

3.6 Latihan 6: Daftar Tugas Sederhana (To-Do List Dinamis)

Tujuan: Membuat aplikasi To-do List yang dapat menambahkan node elemen `<li>` ke dalam `<ul>` secara dinamis menggunakan `document.createElement()` dan menandai tugas selesai via `classList.toggle("selesai")`.

```
// script.js (Latihan 6) const inputTugas = document.getElementById("inputTugas"); const
tambahTugas = document.getElementById("tambahTugas"); const daftarTugas =
document.getElementById("daftarTugas"); const pesanTugas =
document.getElementById("pesanTugas"); tambahTugas.addEventListener("click", function () {
const teks = inputTugas.value.trim(); if (teks === "") { pesanTugas.textContent = "Tulis
tugas terlebih dahulu."; return; } const item = document.createElement("li"); const label =
document.createElement("span"); const selesai = document.createElement("button");
label.textContent = teks + " "; selesai.type = "button"; selesai.textContent = "Tandai
selesai"; selesai.addEventListener("click", function () { item.classList.toggle("selesai");
selesai.textContent = item.classList.contains("selesai") ? "Batalkan selesai" : "Tandai
selesai"; }); item.append(label, selesai); daftarTugas.appendChild(item); inputTugas.value =
""; pesanTugas.textContent = "Tugas ditambahkan."; inputTugas.focus(); });
```



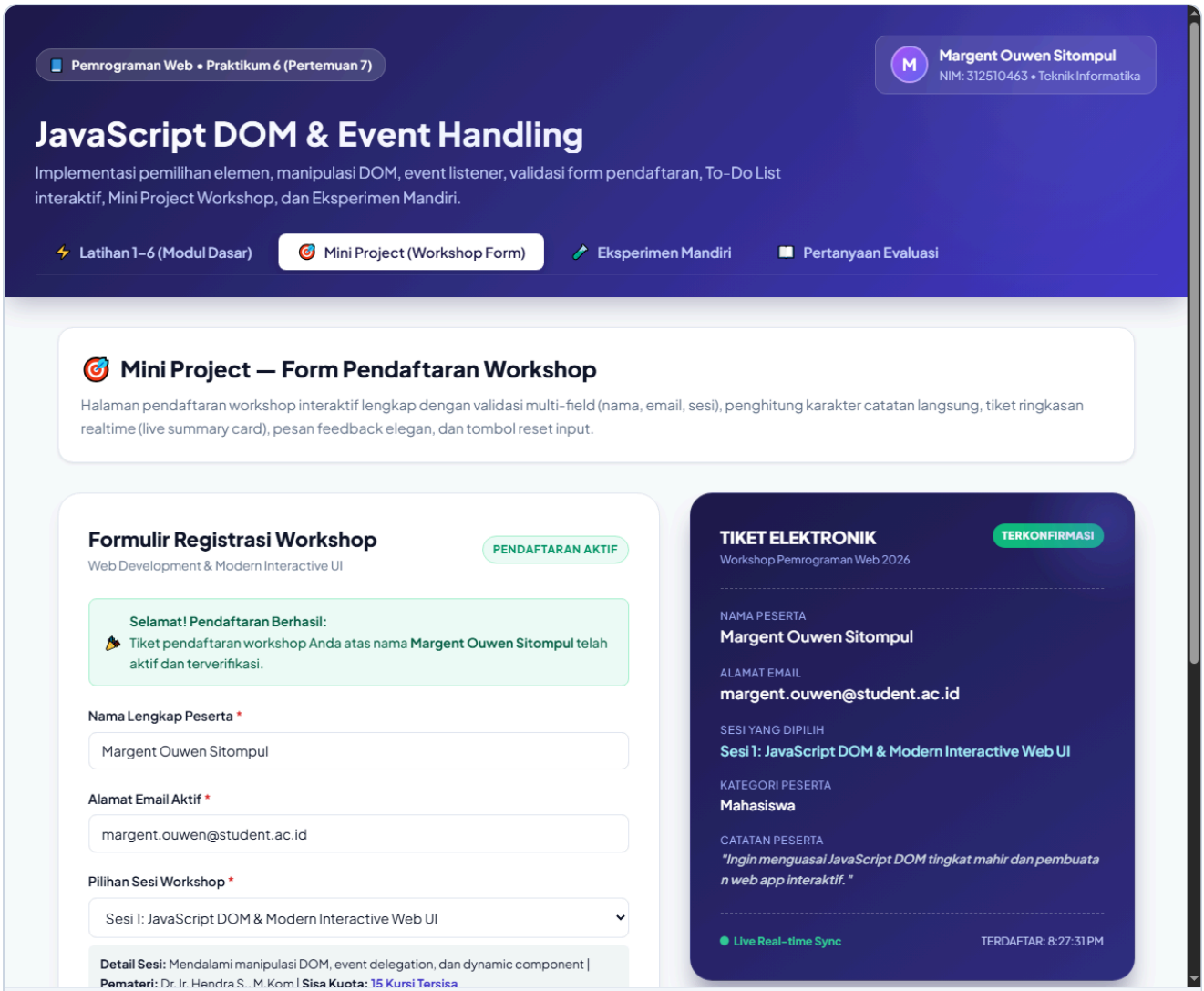
Gambar 3.6: Hasil Pengujian Latihan 6 - To-do List dengan Fitur Tandai Selesai & Tombol Hapus (Eksperimen 1)

BAB IV. MINI PROJECT — FORM PENDAFTARAN WORKSHOP

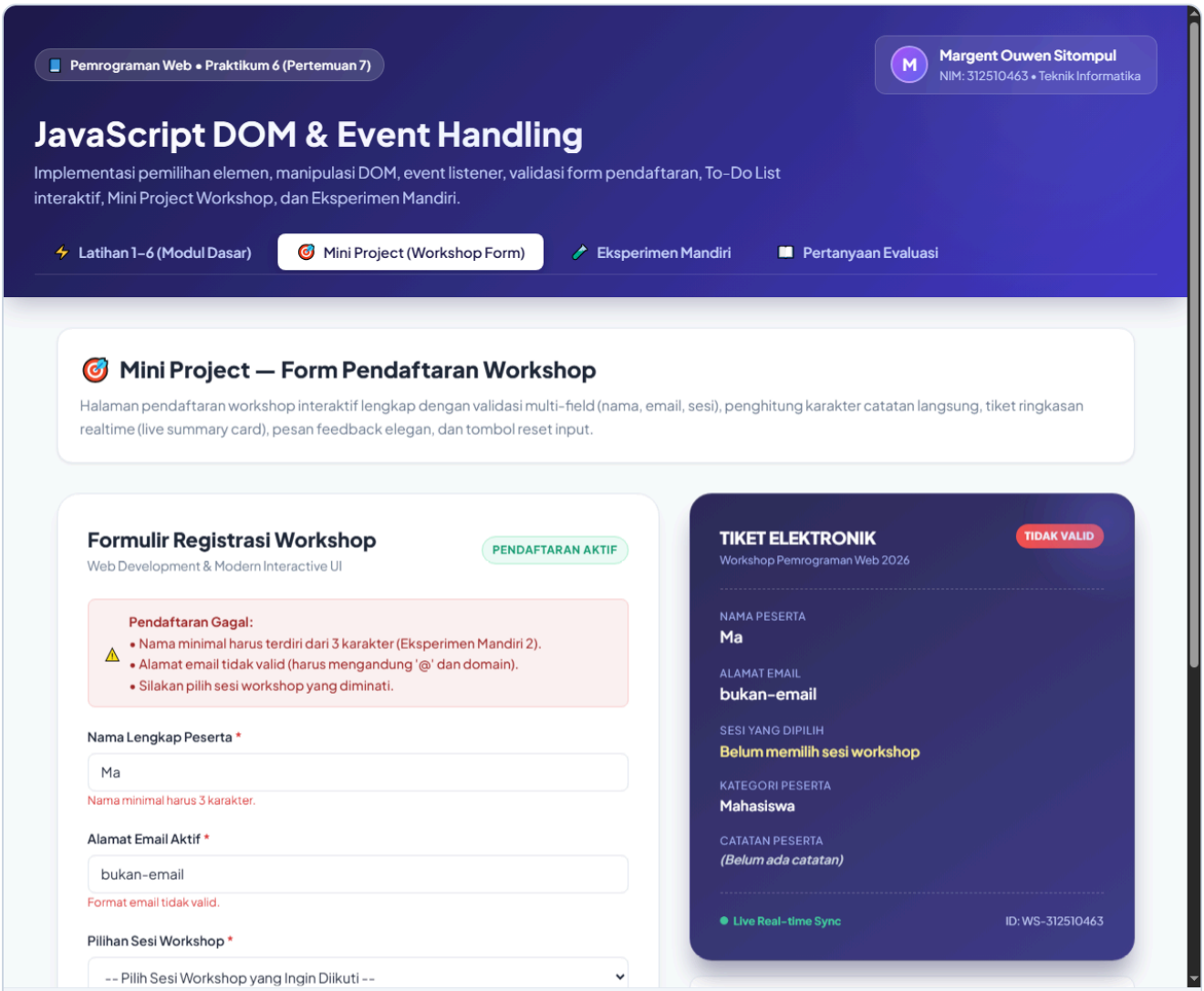
4.1 Deskripsi dan Rancangan Mini Project

Berdasarkan spesifikasi Bagian F Modul Praktikum, Mini Project dikembangkan sebagai aplikasi registrasi workshop interaktif bertajuk **"Web Development & Modern Interactive UI Workshop"**. Aplikasi ini mengintegrasikan seluruh teknik DOM dan event handling yang dipelajari:

- **Validasi Multi-field:** Validasi nama lengkap (minimal 3 karakter), alamat email berformat valid, dan pemilihan sesi workshop.
- **Live Preview Tiket (Realtime Data Synchronization):** Tiket elektronik yang langsung ter-update saat pengguna mengetik nama, email, catatan, atau memilih sesi/kategori.
- **Penghitung Karakter Catatan Langsung:** Indikator panjang teks catatan/harapan peserta (maksimal 150 karakter) secara *live*.
- **Notifikasi Kesalahan & Sukses:** Banner umpan balik visual elegan yang menampilkan rincian error atau status pendaftaran terkonfirmasi.
- **Tombol Reset:** Menyetel ulang seluruh input dan mengembalikan kartu tiket ke status awal (DRAFT).



Gambar 4.1: Tampilan Mini Project Form Pendaftaran Workshop – Kondisi Lengkap & Valid (Tiket Terkonfirmasi)



Gambar 4.2: Tampilan Mini Project Form Pendaftaran Workshop - Kondisi Validasi Gagal (Alert Peringatan Aktif)

BAB V. EKSPERIMEN MANDIRI

5.1 Implementasi dan Analisis 5 Poin Eksperimen Mandiri

1. Tambahkan tombol untuk menghapus satu item dari To-do List

Solusi: Pada fungsi pembuat item to-do, ditambahkan elemen tombol `<button>` berlabel "Hapus" yang memiliki event listener `item.remove()` (atau `daftarTugas.removeChild(item)`). Ketika diklik, node `<li>` terkait langsung dihilangkan dari pohon DOM.

2. Ubah validasi agar nama minimal terdiri dari tiga karakter

Solusi: Kondisi validasi nama diperluas dari `nama === ""` menjadi:

```
if (nama === "") { pesanForm.textContent = "Nama wajib diisi."; } else if (nama.length < 3)
{ pesanForm.textContent = "Nama minimal harus terdiri dari 3 karakter."; }
```

3. Tambahkan event change pada dropdown sesi workshop dan tampilkan sesi yang dipilih

Solusi: Dropdown `<select id="wsSesi">` dipasangi event listener `change`. Saat pengguna memilih opsi baru, JavaScript membaca nilai `wsSesi.value` serta atribut data (deskripsi, pemateri, sisa kuota) dan menampilkannya langsung ke kotak informasi sesi.

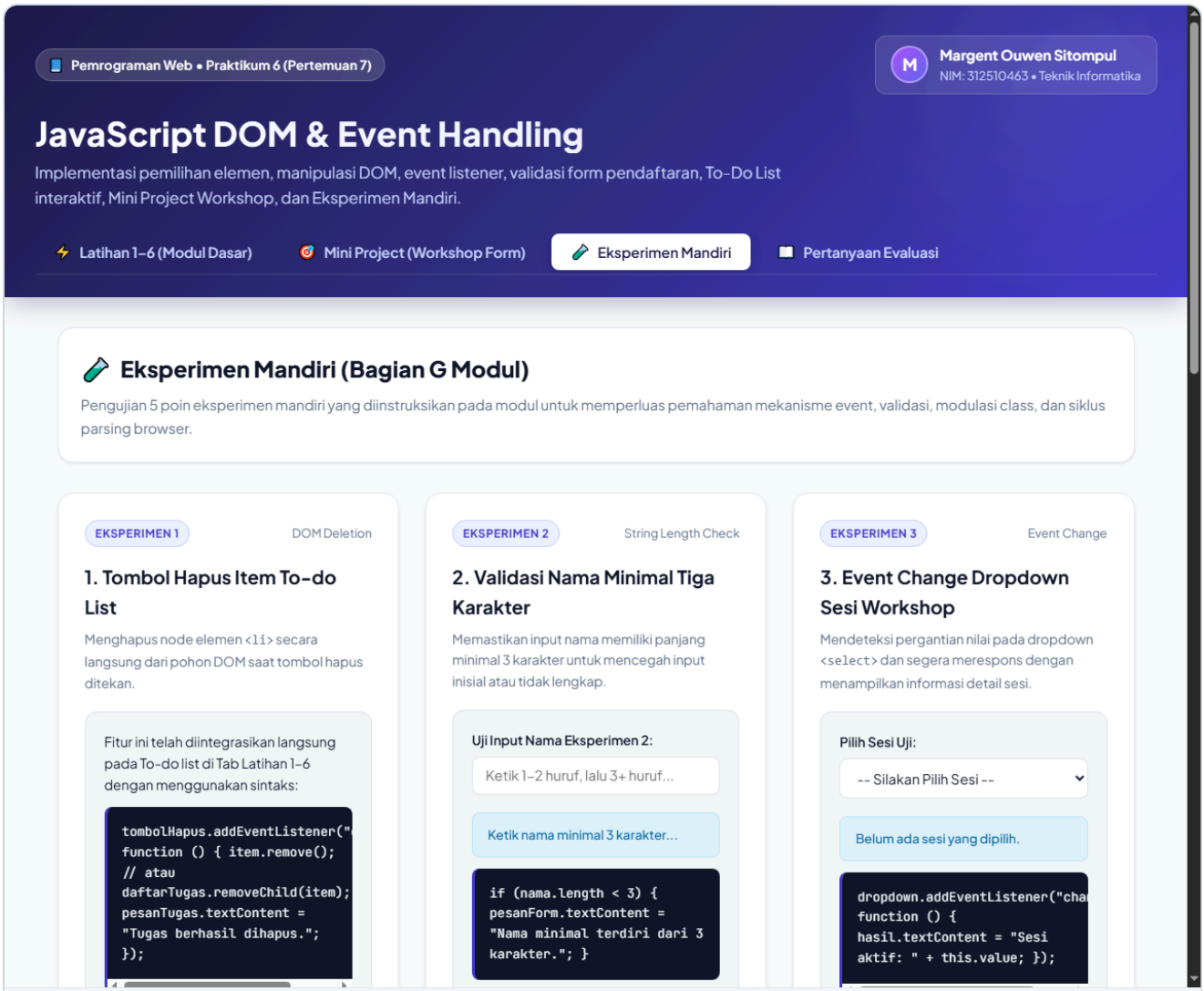
4. Buat tombol untuk menyembunyikan dan menampilkan paragraf menggunakan class CSS

Solusi: Dibuat class CSS `.hidden-element { display: none !important; }` dan tombol aksi yang memanggil `paragraf.classList.toggle("hidden-element")`. Ketika class ditambahkan, paragraf disembunyikan; saat dicopot, paragraf tampil kembali.

5. Pindahkan script tanpa defer ke bagian head. Amati kemungkinan masalah dan jelaskan penyebabnya

Hasil Pengamatan & Penyebab:
Ketika tag `<script src="script.js"></script>` ditaruh di dalam tag `<head>` **tanpa atribut `defer`**:

- Browser web melakukan eksekusi skrip secara *synchronous* saat membaca bagian head, sebelum dokumen HTML bagian body selesai dibaca (diparsing).
- Fungsi pemilih elemen seperti `document.getElementById("judul")` akan mengembalikan nilai `null` karena elemen HTML dengan ID tersebut belum ada di dalam DOM.
- Ketika skrip mencoba mengeksekusi `judul.textContent = ...`, browser akan melempar pesan kesalahan runtime: `Uncaught TypeError: Cannot set properties of null (setting 'textContent')` dan seluruh skrip di bawahnya akan berhenti bekerja.
- **Solusi Standar:** Menggunakan atribut `defer` pada tag script di bagian head, atau menaruh tag script sebelum tag penutup `</body>`, atau membungkus kode dalam `document.addEventListener("DOMContentLoaded", ...)`.



Gambar 5.1: Hasil Pengujian Tab Eksperimen Mandiri (1–5) pada Halaman Web Lab6Web

BAB VI. JAWABAN PERTANYAAN EVALUASI

Pertanyaan 1: Apa yang dimaksud dengan DOM dan apa manfaatnya?

Jawaban:

Document Object Model (DOM) adalah antarmuka pemrograman berstandar W3C yang memodelkan dokumen HTML sebagai pohon struktur berorientasi objek (*tree of objects*). Setiap tag, atribut, dan teks diwakili sebagai node objek yang memiliki properti dan metode.

Manfaat DOM:

- Memungkinkan JavaScript memodifikasi struktur, teks, dan elemen HTML secara dinamis tanpa reload halaman.
- Mendukung interaktivitas tinggi pengguna melalui event handling (klik, ketik, drag, scroll).
- Memudahkan pembaruan antarmuka secara modular, cepat, dan responsif.

Pertanyaan 2: Apa perbedaan getElementById(), querySelector(), dan querySelectorAll()?

Jawaban:

Perbedaan ketiganya terletak pada jenis selector yang diterima, nilai kembali, dan performanya:

Metode	Argumen Selector	Nilai Return	Karakteristik
`getElementById(id)`	Hanya string nama ID (tanpa tanda #)	1 objek Element tunggal atau `null`	Performa paling cepat karena browser mengindeks ID secara khusus.
`querySelector(sel)`	Selector CSS apa pun (`#id`, `.class`, tag, attribute)	1 elemen pertama yang cocok atau `null`	Fleksibel untuk memilih elemen bersarang spesifik.
`querySelectorAll(sel)`	Selector CSS apa pun	Koleksi `NodeList` statis	Mengambil semua elemen yang cocok, mendukung method `.forEach()`.

Pertanyaan 3: Apa fungsi addEventListener() dan sebutkan tiga contoh event?

Jawaban:

Fungsi `addEventListener()` adalah metode standar pada objek DOM EventTarget untuk mendaftarkan fungsi penanganan (*event listener/callback*) yang akan dipanggil saat suatu event tertentu terjadi pada elemen tersebut. Keunggulannya adalah dapat memasang beberapa penangan berbeda pada elemen yang sama tanpa menimpa fungsi lain.

Tiga Contoh Event:

- `click`: Terpicu saat tombol mouse ditekan atau layar sentuh disentuh pada suatu elemen.
- `input`: Terpicu secara instan setiap kali isi teks input/textarea berubah (berguna untuk character counter).
- `submit`: Terpicu saat formulir dikirimkan oleh pengguna.

Pertanyaan 4: Mengapa event.preventDefault() digunakan pada event submit form?

Jawaban:

Secara bawaan (*default behavior*), pengiriman formulir HTML akan memicu browser untuk memuat ulang halaman atau berpindah ke URL atribut `action`. Pemanggilan `event.preventDefault()` digunakan untuk membatalkan perilaku bawaan tersebut, sehingga proses validasi formulir dan manipulasi DOM dapat diselesaikan sepenuhnya oleh JavaScript di sisi klien tanpa kehilangan status data atau menyebabkan kedipan reload halaman.

Pertanyaan 5: Apa perbedaan textContent dan innerHTML dari sisi penggunaan dan keamanan?

Jawaban:

Aspek	`textContent`	`innerHTML`
Penggunaan	Menyisipkan atau membaca string sebagai teks murni tanpa mengurai tag HTML.	Menyisipkan string yang akan di-parse oleh browser menjadi elemen HTML nyata.
Keamanan	**Aman.** Tidak dapat disusupi kode berbahaya karena tag diperlakukan sebagai karakter teks literal.	**Rentan XSS.** Jika teks pengguna disisipkan langsung, penyerang dapat menyuntikkan skrip jahat.
Kinerja	Lebih cepat karena tidak memicu parser HTML browser.	Lebih lambat karena browser harus mem-parsing markup string menjadi node DOM.

Pertanyaan 6: Bagaimana classList.toggle() membantu mengubah tampilan elemen?

Jawaban:

`classList.toggle("namaKelas")` berfungsi sebagai sakelar (*toggle switch*): jika kelas tersebut belum ada pada elemen, method akan menambahkannya; jika sudah ada, method akan menghapusnya. Hal ini menghemat kode karena pengembang tidak perlu menulis logika pengecekan kondisi manual `if (el.classList.contains(...))`. Fitur ini sangat bermanfaat untuk tombol tema (dark mode), accordion, dan penandaan status selesai pada to-do list.

Pertanyaan 7: Sebutkan minimal tiga skenario pengujian validasi form!

Jawaban:

Berikut adalah 5 skenario pengujian yang telah diuji dan divalidasi pada praktikum ini:

1. **Semua Input Kosong:** Pengguna langsung menekan submit; sistem memunculkan pesan *"Nama wajib diisi."*.
2. **Format Email Tidak Valid:** Pengguna memasukkan nama valid tetapi email tidak memiliki domain/simbol '@'; sistem memunculkan pesan *"Masukkan email yang valid."*.
3. **Usia di Bawah 17 Tahun:** Pengguna memasukkan usia 15 tahun; sistem menolak dengan pesan *"Usia harus berupa bilangan bulat 17–100."*.
4. **Usia Bukan Bilangan Bulat:** Pengguna memasukkan nilai pecahan (misal 21.5); sistem menolak.
5. **Semua Data Valid:** Pengguna mengisi nama lengkap, email valid, dan usia 21 tahun; sistem memunculkan pesan sukses *"Validasi berhasil. Form siap diproses."* dan mereset form.

BAB VII. KESIMPULAN

Berdasarkan serangkaian latihan, pengembangan mini project, dan eksperimen mandiri yang telah diselesaikan pada Praktikum 6 (Pertemuan 7) Pemrograman Web, dapat disimpulkan bahwa:

1. **Document Object Model (DOM)** merupakan pilar utama pengembangan antarmuka web modern yang memungkinkan JavaScript mengakses, memodifikasi, dan menghapus elemen HTML secara terprogram.
2. Penggunaan `addEventListener()` memberikan fleksibilitas tinggi dalam menangani berbagai event (click, input, change, submit) secara modular dan terpisah dari markup HTML.
3. Validasi sisi klien dengan `event.preventDefault()` sangat penting untuk memberikan umpan balik langsung (*instant feedback*) kepada pengguna sebelum data dikirimkan ke server, sehingga meningkatkan efisiensi dan pengalaman pengguna (User Experience).
4. Penerapan `textContent` terbukti lebih aman dibandingkan `innerHTML` dalam menangkal kerentanan Cross-Site Scripting (XSS).
5. Atribut `defer` pada tag skrip di dalam `<head>` merupakan praktik terbaik (*best practice*) agar seluruh pohon DOM selesai diparsing oleh browser sebelum skrip dieksekusi, menghindari kesalahan fatal `null reference`.

Jakarta, 27 Oktober 2026

Praktikan,

Margent Ouwen Sitompul

NIM. 312510463